StartupForge AI

Implementation and final integration plan

Three independent app modules | Team size 3 | 7 October 2026

The team will research, implement, and verify three app modules in parallel. Operational integration happens only after all three modules finish development. Before coding, the team agrees on data schemas, API contracts, and test fixtures so that the finished modules can connect without redesign.

Module ownership

Member	App module	Complete feature ownership
1	Idea Intelligence	Idea entry, structured extraction, external research, RAG, analysis, evidence critic, sources and analysis UI.
2	Pitch and Visual Studio	Pitch narrative, slide content, PPTX creation, poster content, templates, previews and output editing.
3	Report and Project Workspace	Formal report generation, DOCX export, project save/load, context versions, navigation, jobs and download infrastructure.

Each member owns the frontend feature, backend routes and services, prompts, module configuration, and verification for their module. Member 3 maintains shared infrastructure and coordinates the final assembly; each member fixes compatibility issues inside their own module.

Development model

1. Research the assigned module's methods, libraries, limits, and implementation approach. Generating project PPTs, posters, and reports belongs to implementation, not this research phase.
2. Agree on the shared contracts and complete sample project data. Sharing specifications and fixtures does not require combining running modules.
3. Develop and test all three modules independently with local adapters and sample data.
4. Freeze the completed modules after their independent demonstrations pass.
5. Assemble the application, replace local adapters with shared storage, and run the final integration checks.

Minimum deliverables

A reviewed, evidence-supported project context; pitch deck content and a usable PPTX; downloadable poster content; a formal DOCX report; and a workspace that saves projects and provides downloads. PDF report export and fully rendered poster graphics may be added through the same export interfaces after their dependencies are available.

The final application uses one frontend and one FastAPI backend. The API catalogue in this document defines 26 application endpoints under `/api/v1`. Internal search, embedding, retrieval, and critic functions remain implementation details inside their owning module.

1 Module research and implementation

Member 1: Idea Intelligence

Research idea extraction, source quality, search APIs, chunking, embeddings, retrieval, grounded analysis, and critic prompts. Keep the prototype to one chosen search provider, one retrieval approach, bounded source ingestion, and at most one critic revision pass.

Implement the idea entry and analysis screens, structured extraction, research and RAG, competitor and feasibility findings, risks, assumptions, source references, and the critic. Preserve the initial analysis, critic findings, and reviewed final analysis. Unsupported claims must be removed, qualified, or recorded as gaps.

Independent demonstration: submit an idea, inspect extracted fields, run analysis, and inspect sources and critic feedback without starting either teammate's server. Use a local job store and a fixture project repository.

Owned folders: `backend/app/modules/intelligence/` and `frontend/src/modules/intelligence/`.

Member 2: Pitch and Visual Studio

Research pitch structure, converting context into slide content, PPTX tooling, templates, text fitting, poster structure, and consistency checks. Artifact production starts during implementation.

Implement the pitch and poster screens, structured content generation, editing, templates, actual PPTX rendering, and a poster-content export. Check factual consistency and citations against the supplied context. Preview structured content in the frontend; pixel-exact previews of rendered slides require a separate rendering capability.

Independent demonstration: load the shared sample context, create and edit slide/poster content, and download a usable PPTX and poster text without calling Member 1's server.

Owned folders: `backend/app/modules/visual_studio/` and `frontend/src/modules/visual_studio/`, including their templates and renderers.

Member 3: Report and Project Workspace

Research formal report structure, long-form generation, citations, DOCX/PDF export, simple persistence, module registration, and job handling.

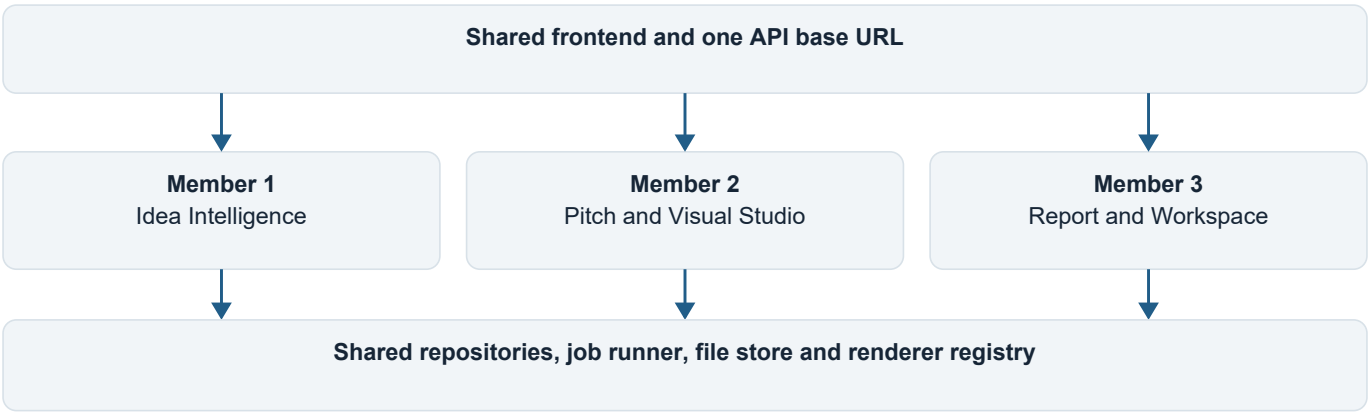
Implement report generation, editing and export; project save/load; immutable context versions; shared navigation; common job and artifact APIs; and file downloads. Build the shared shell against placeholder module screens and mock service implementations until final integration.

Independent demonstration: save/reopen the sample project, preview and edit report sections, and download an openable DOCX. Keep the workspace small and focused on the capstone workflow.

Owned folders: `backend/app/modules/reports/`, `frontend/src/modules/reports/`, `backend/app/core/`, and `frontend/src/app/`.

Member 3 maintains shared schema files after team agreement. All members supply their module's startup instructions, configuration example, fixtures, API examples, and dependency list. Changes to shared contracts require agreement and updated fixtures.

2 How the completed modules integrate



During independent development

Each member runs a standalone FastAPI app and a feature UI. A development harness supplies the common project, job, artifact, and file interfaces using local implementations. Modules 2 and 3 receive a complete sample ProjectContext; they never need a live intelligence server to develop their generation features.

Each module exports a mountable frontend route, a FastAPI router, and callable application services. It must accept shared dependencies through interfaces rather than importing another member's implementation. Local harness routes are excluded from the final application.

At final assembly

Member 3 mounts all feature routes in the shared frontend and registers all feature routers in one FastAPI app. Common routes are registered once. Shared repository, job-runner, file-store, and renderer implementations replace the local adapters through dependency injection.

The frontend uses one API base URL. The final backend invokes module services directly for the combined workflow; it does not need to make HTTP requests to itself. Each feature API remains independently usable from the frontend.

Runtime data flow

1. Create a project and submit the idea for extraction. The user can correct the structured fields before analysis.
2. Run research, analysis, and critique. Member 1 returns an uncommitted ContextDraft with evidence and reviewed findings.
3. The workspace commits the draft as an immutable ProjectContext version through the context API.
4. Send that complete snapshot to pitch, poster, and report generation. These three jobs may run concurrently.
5. Each generator registers structured content and exported files through the shared artifact and file interfaces.
6. The workspace shows previews, editing controls, job status, and downloads. Editing content and rendering a new file use separate versioned operations.

A combined run endpoint provides the original single-submission experience. It performs extraction and analysis, commits one reviewed context, then starts the requested output jobs. The separate endpoints also support the step-by-step UI.

3 Shared data and service contracts

Use shared typed models with `schema_version: "1.0"`. IDs are opaque strings, timestamps use UTC ISO 8601, and context/content versions are positive integers. A newly created project has `latest_context_version: 0` until its first context is committed.

Model	Required fields and meaning
IdeaSpec	original_text, project_name, problem, solution, target_users[], domain, assumptions[], constraints[]. Mark inferred or unknown fields explicitly.
Source / EvidenceItem	Source: source_id, title, url, retrieved_at. Evidence: evidence_id, source_id, excerpt, optional locator. Every reference must resolve.
AnalysisReport	Named sections for summary, competitors, feasibility, risks and gaps. Claims have claim_id, text, evidence_ids[] and kind: supported, assumption or unknown.
CritiqueReport	review_status, issues[] with severity/claim reference/reason, revision_summary, unresolved_gaps[]. Preserve initial and final analysis.
ContextDraft	idea, sources[], evidence[], initial_analysis, final_analysis, critic, warnings[]. This is an uncommitted analysis result.
ProjectContext	ContextDraft fields plus schema_version, project_id, context_version, created_at and content_hash, assigned when committed.
Job	job_id, operation, project_id, context_ref when applicable, status, stage, progress, timestamps, typed result or result_url, error, child_job_ids[].
Artifact	artifact_id, project_id, context_version, context_hash, kind, content_revision, producing_job_id, files[] and timestamps. Public records contain file IDs and URLs, not local disk paths.

`review_status` is `ready`, `ready_with_gaps`, or `blocked`. Generators accept the first two, carrying warnings and evidence gaps into the output. A blocked context is rejected. Committed contexts are immutable; a correction or new analysis creates another version.

The workspace assigns a deterministic hash to each committed snapshot. Generation receives the entire snapshot inline and verifies its identity against the repository. During standalone development the same repository interface is backed by the agreed fixture. Jobs pin an exact version and never silently switch to the newest version.

Shared interfaces

- ProjectRepository: create/list/read projects and atomically commit/read context versions.
- JobRepository and JobRunner: submit, track and retrieve asynchronous work and typed results.
- ArtifactRepository: store manifests and immutable structured-content revisions.
- FileStore: register and read generated bytes by file ID.
- RendererRegistry: dispatch artifact kind and format to Member 2's or Member 3's renderer.

Member 3 implements the final shared adapters. Members 1 and 2 use compatible local adapters in their standalone harnesses. Module services depend only on the interface definitions and shared models.

4 Workspace and context API

All routes in the catalogue are relative to `/api/v1`. JSON is used unless a route returns file bytes. Owner: Member 3. `id` in a route means the corresponding project ID.

Method	Route	Request	Success response
GET	/health	None	200: readiness, loaded modules, supported export formats and schema version.
POST	/projects	display_name, idea_text	201: Project with project_id, revision=1 and latest_context_version=0.
GET	/projects	Optional cursor, limit	200: items[] and next_cursor. Default limit 20; maximum 100.
GET	/projects/{id}	None	200: Project metadata, revision, latest context reference and timestamps.
PATCH	/projects/{id}	expected_revision and metadata changes: display_name and/or draft_idea_text	200: updated Project. Metadata edits do not change existing contexts.
POST	/projects/{id}/contexts	expected_latest_version, context: ContextDraft	201: immutable ProjectContext with server-assigned version, timestamp and hash.
GET	/projects/{id}/contexts	Optional cursor, limit	200: version summaries, review status, timestamps and next_cursor.
GET	/projects/{id}/contexts/{version}	None	200: exact ProjectContext, including sources, evidence, analysis and critic.
GET	/projects/{id}/context	None	200: latest committed ProjectContext; 404 if none exists.

Atomic context commit

For the first commit use `expected_latest_version: 0`. The server validates the entire ContextDraft, including evidence references, then compares the expected version with the stored latest version. It assigns the next version and updates the latest pointer in one transaction. A mismatch returns 409 and leaves existing snapshots unchanged.

An edited raw idea remains draft input until intelligence processes it. Editing a project's metadata or draft idea does not make old outputs current for the new idea. The UI retains each output's source context label.

A failed analysis never replaces the last committed context. A failed context commit leaves the analysis result available, so the UI can refresh the project and explicitly decide whether to rerun or commit after resolving the conflict.

Project metadata

Project contains project_id, display_name, draft_idea_text, revision, latest_context_version, created_at and updated_at. Its metadata revision is separate from context_version and artifact content_revision.

5 Intelligence, jobs and combined workflow API

All routes use the `/api/v1` prefix. Every asynchronous POST requires an `Idempotency-Key` header and returns a JobReceipt: job_id, status_url, operation and project_id. Poll status_url until a terminal status is reached.

Method	Route	Request	Success response and owner
POST	/intelligence/extractions	project_id, idea_text	202 JobReceipt. Job result is IdeaSpec. Owner 1.
POST	/intelligence/analyses	project_id, idea: IdeaSpec, research_options	202 JobReceipt. Result references an AnalysisResult. Owner 1.
GET	/intelligence/analyses/{analysis_id}	None	200: analysis_id, project_id, ContextDraft, stages and warnings. Owner 1.
GET	/jobs/{job_id}	None	200: Job with progress, stage, result/result_url or structured error. Owner 3.
GET	/projects/{id}/jobs	Optional cursor, limit, status	200: jobs[] and next_cursor. Owner 3.
POST	/projects/{id}/runs	expected_latest_version, idea_text, outputs[], output_options	202: parent JobReceipt for the combined workflow. Owner 3; connected at final assembly.

`research_options` supports max_sources, language and optional region. The team fixes supported limits during research and publishes them in the module configuration. Search, document processing, embeddings, retrieval, analysis and critique occur within the analysis operation; they do not require separate public endpoints.

Jobs and results

Normal job states: `queued`, `running`, `succeeded`, `failed`. The combined parent job may also end as `partial_success`. Progress is a value from 0 to 100; stage names describe the actual current operation. A succeeded job must have a valid result or resolvable result URL.

The analysis job stores its result before it reports success. GET analysis returns the complete uncommitted draft; the workspace commits it using the context endpoint. Extraction and analysis jobs expose useful errors without creating a successful-looking result.

Combined run semantics

Outputs may contain `pitch_deck`, `poster`, and `report`. The coordinator calls the same module services used by the feature routes. It extracts the idea, runs intelligence, atomically commits against expected_latest_version, and passes the resulting snapshot to all requested output services.

If intelligence or the context commit fails, no output jobs start. If one output fails, successful outputs remain available and the parent reports partial_success with successful_outputs[], failed_outputs[] and child_job_ids[]. The parent succeeds only when every requested output succeeds. Member 3 wires this coordinator after the completed modules are assembled.

6 Presentation, poster and report API

All routes use `/api/v1`. Generation POSTs require `Idempotency-Key`. Each request carries a complete ProjectContext snapshot and options; no generator calls the intelligence module to obtain missing facts.

Method	Route	Request	Success response and owner
GET	/studio/templates	Optional kind: pitch_deck or poster	200: template IDs, names, supported options and formats. Owner 2.
POST	/studio/pitch-decks	context: ProjectContext, options: template_id, slide_count, tone	202 JobReceipt. Produces structured deck content and an initial PPTX. Owner 2.
POST	/studio/posters	context: ProjectContext, options: template_id, audience	202 JobReceipt. Produces poster content and an initial TXT export. Owner 2.
GET	/reports/templates	None	200: report templates, required sections and supported formats. Owner 3.
POST	/reports/generations	context: ProjectContext, options: template_id, formats[]	202 JobReceipt. Produces structured report content and requested exports. Owner 3.

All template_id values come from the relevant template endpoint. Each template publishes option bounds; invalid values return 422. For the minimum implementation, formats include PPTX for decks, TXT for posters and DOCX for reports. Report PDF and poster graphics can be enabled by the renderer registry without introducing a new generation endpoint.

Structured editable content

Kind	Content schema
pitch_deck	title and slides[]. Each slide has slide_id, title, body_blocks[], speaker_notes, source_ids[] and evidence_ids[].
poster	title and sections[]. Each section has section_id, heading, body, source_ids[] and evidence_ids[].
report	title and sections[]. Each section has section_id, heading, paragraphs[], source_ids[] and evidence_ids[]; bibliography is built from the context sources.

A successful generation job returns artifact_ids[] and file_ids[]. Structured content is stored as revision 1. Artifact metadata records the exact context version/hash, and every initial export records which content revision it rendered.

Consistency and previews

Generators may adapt narrative and wording while preserving the project's facts, assumptions and source references. Gaps in the context remain visible. Output checks compare claims and citations with the supplied snapshot before the job succeeds.

The frontend uses the structured content API to render its preview/editor. Exported files are separate outputs. If a requested format is unavailable, return UNSUPPORTED_FORMAT with supported_formats rather than silently substituting another format.

7 Artifact editing, rendering and download API

All routes use `/api/v1`. Member 3 owns the shared endpoints and persistence. Members 2 and 3 supply their content validators, preview components and renderers.

Method	Route	Request	Success response
GET	/projects/{id}/artifacts	Optional context_version, kind, cursor, limit	200: artifact manifests[] and next_cursor.
GET	/artifacts/{artifact_id}	None	200: manifest, current content revision, context reference and available file records.
GET	/artifacts/{artifact_id}/content	Optional revision; defaults to latest	200: kind, content_revision and typed structured content. Used for preview and editing.
PUT	/artifacts/{artifact_id}/content	base_revision, complete content object	200: new content_revision and validation findings. Creates an immutable revision.
POST	/artifacts/{artifact_id}/renders	content_revision, formats[]; Idempotency-Key header	202 JobReceipt. Renders the exact requested revision through the owning module's renderer.
GET	/files/{file_id}/download	None	200: file bytes with Content-Type and Content-Disposition filename.

Editing and rerendering

The editor reads the current revision, submits the complete edited content with base_revision, and receives a new revision. A stale base_revision returns 409; the existing content remains unchanged. A content-shape or citation-reference failure returns 422.

Editing content does not overwrite prior files. The UI marks old exports as belonging to their earlier content revision and offers a render action for the new revision. A render job pins its revision at submission; edits made while it runs do not alter that job's input.

Changes to actual project facts must first create a new reviewed project context. Content editing is for wording and presentation within the selected context. The owning module checks edited content for consistency and reports conflicts before rendering.

Artifact and file identity

Each file record includes file_id, artifact_id, context_version, content_revision, format, media_type, filename, size_bytes, created_at and download_url. The file store resolves IDs to internal storage locations. Download endpoints never accept user-supplied filesystem paths.

Old outputs remain downloadable after a new context version is created. A newer context does not silently rewrite an existing artifact. Regeneration from that context creates a new artifact whose provenance is explicit.

The shared render endpoint selects Member 2's renderer for pitch_deck/poster and Member 3's renderer for report. It uses the same renderer implementations as initial generation.

8 Request and response examples

The examples use readable IDs. Actual implementations may generate UUIDs. Every example route includes the full `/api/v1` prefix.

Create and extract

```
POST /api/v1/projects
{"display_name":"Campus Meals","idea_text":"Affordable campus meal plans"}

201
{"project_id":"p1","revision":1,"latest_context_version":0}

POST /api/v1/intelligence/extractions
Idempotency-Key: p1-extract-01
{"project_id":"p1","idea_text":"Affordable campus meal plans"}

202
{"job_id":"j1","operation":"extract_idea","project_id":"p1",
 "status_url":"/api/v1/jobs/j1"}
```

GET the status URL. On success, result contains a complete IdeaSpec. The UI permits correction, then sends that object as `idea` to POST `/api/v1/intelligence/analyses` with a new idempotency key. The successful analysis job returns a result_url pointing to its AnalysisResult.

Commit the reviewed context

```
POST /api/v1/projects/p1/contexts
{
  "expected_latest_version": 0,
  "context": "<complete ContextDraft from AnalysisResult>"
}
```

The placeholder above represents a JSON object, not a literal string. The server returns the complete committed snapshot with project_id, context_version 1, created_at, content_hash, idea, sources, evidence, analyses, critic and warnings.

Generate outputs in parallel

```
POST /api/v1/studio/pitch-decks
Idempotency-Key: p1-v1-deck-01
{
  "context": "<complete committed ProjectContext object>",
  "options": {"template_id":"basic-pitch","slide_count":10,
              "tone":"professional"}
}
```

Send the same snapshot to POST `/api/v1/studio/posters` and POST `/api/v1/reports/generations` with distinct keys and appropriate options. The example template ID is illustrative; use IDs returned by the template APIs. Poll each job, read its artifact manifest, and use GET `/api/v1/files/{file_id}/download` for exported files.

For a single-click run, POST `/api/v1/projects/p1/runs` with idea_text, expected_latest_version and outputs. The coordinator performs these steps and reports its child jobs.

9 Errors, retries and version rules

Use one error envelope across all modules. The same structure appears in failed Job records. Include a request ID so logs from the frontend, router and worker can be matched.

```
{
  "error": {
    "code": "CONTEXT_VERSION_CONFLICT",
    "message": "The project has a newer context version.",
    "details": {"expected": 1, "actual": 2},
    "retryable": false,
    "request_id": "req-123"
  }
}
```

HTTP status	Meaning and expected handling
200	Read, edit or file download succeeded.
201	Project or immutable context created.
202	Job accepted; use its status URL to determine completion.
400	Malformed request syntax.
404	Project, context, job, analysis, artifact, revision or file does not exist.
409	Context/metadata/content revision conflict, snapshot mismatch, or reused idempotency key with different input. Refresh and resolve explicitly.
422	Schema violation, invalid reference, blocked context, unsupported option or unavailable export format.
429	Provider or application rate limit. The error states whether and when a retry is appropriate.
502 / 503	Required upstream service or local capability failed or is unavailable. Expose a useful operation-specific error.
500	Unexpected internal failure; preserve the request ID and log details on the server.

Idempotency

Require Idempotency-Key for extraction, analysis, combined runs, output generation and rendering. Scope it by operation and project/artifact. Store a fingerprint of the normalized input. Repeating the same key and input returns the original JobReceipt; changing the input with that key returns 409. Retrying a failed operation as a new attempt uses a new key.

Failure isolation

An asynchronous provider failure appears in the Job record even though the submission returned 202. A failed output job does not remove another module's successful outputs. A rendering failure preserves editable content and existing files. Context commits and content revisions are atomic.

Compatibility

Freeze schema version 1.0 before development. Validate the same fixtures in every module. Changing a field, route, enum or required option means updating the shared contract and fixtures with team agreement. Pin a compatible dependency set at final assembly and run contract checks before runtime tests.

10 Final integration procedure and acceptance

Gate before assembly

Each member completes their research, implementation and independent verification. Their module must run with the other two servers stopped. Deliver its feature UI, router, services, dependency list, environment example, startup instructions, fixtures, and successful/error API examples. Commit or tag that completed version.

Assemble only after all three modules pass

1. Combine the completed module folders and agree on one compatible frontend/backend dependency set. Resolve duplicate package versions and configuration names.
2. Register the three frontend routes and feature routers. Mount the common workspace, job, artifact and file routes once; remove development harness routes from the final app.
3. Inject the shared ProjectRepository, JobRepository/Runner, ArtifactRepository, FileStore and RendererRegistry. Preserve the public contracts used by local adapters.
4. Connect create/open project, extraction, analysis-result review and context commit. Confirm the committed context can be reopened unchanged.
5. Connect pitch, poster and report generation to the same saved context snapshot. Wire job polling, preview/editing, rendering and download controls.
6. Connect the combined run coordinator to the real module services. Verify partial failures and child-job reporting.
7. Run the acceptance checks below. Every member resolves problems in their own module; Member 3 coordinates shared wiring and storage issues.

Acceptance checks

Check	Pass condition
Complete workflow	A new idea reaches reviewed analysis and all required outputs without manual copying between modules.
Parallel output jobs	Deck, poster and report consume the same context version and can run independently.
Source traceability	All evidence/source IDs resolve, and unsupported fixture claims are flagged or qualified.
Persistence	Save/reopen retains context versions, content revisions, job results and file links.
Editing	A stale edit receives 409; an accepted edit creates a new content revision; rendering uses the selected revision.
Context changes	Reanalysis creates a new context version. Old artifacts retain their original version and remain downloadable.
Exports	PPTX and DOCX open successfully; poster content downloads; enabled optional formats also open.
Failure isolation	An intentionally failed export preserves other successful outputs and reports a useful error.
Retry behavior	A duplicate request with the same key does not create another job or artifact.

Scope of the API catalogue

The 26 routes cover project persistence, context versions, extraction/analysis, jobs, the combined workflow, generation, templates, content editing, rendering and downloads. Authentication, billing, collaboration, project deletion and job cancellation are outside this prototype contract. They can be designed later if scope expands.